while (*condition*)

action

for (*set_counter* ; *test_counter* ; *increment_counter*)

action


Awk


Linux

- if (*expression*) *action1*; [else *action2*]

expr ? *action1* : *action2*

grade = (avg >= 65) ? "Pass" : "Fail"

array[*subscript*] = *value*

for (*variable* in *array*)

    do something with array[*variable*]

Awk Function	Description
cos(*x*)	Returns cosine of *x* (*x* is in radians).
exp(*x*)	Returns *e* to the power *x*.
int(*x*)	Returns truncated value of *x*.
log(*x*)	Returns natural logarithm (base-*e*) of *x*.
sin(*x*)	Returns sine of *x* (*x* is in radians).
sqrt(*x*)	Returns square root of *x*.
atan2(*y*,*x*)	Returns arctangent of *y*/*x* in the range -π to π.
rand( )	Returns pseudo-random number *r*, where 0 <= *r* < 1.
srand(*x*)	Establishes new seed for **rand( )**. If no seed is specified, uses time of day. Returns the old seed.

Linux
Awk

- if (*expression*) *action1*; [else *action2*]

expr ? *action1* : *action2*

grade = (avg >= 65) ? "Pass" : "Fail"

while (*condition*)

action

for (*set_counter* ; *test_counter* ; *increment_counter*)

action

array[*subscript*] = *value*

for (*variable* in *array*)

do something with array[*variable*

Awk Function	Description
cos(*x*)	Returns cosine of *x* (*x* is in radians).
exp(*x*)	Returns *e* to the power *x*.
int(*x*)	Returns truncated value of *x*.
log(*x*)	Returns natural logarithm (base-*e*) of *x*.
sin(*x*)	Returns sine of *x* (*x* is in radians).
sqrt(*x*)	Returns square root of *x*.
atan2(*y*,*x*)	Returns arctangent of *y*/*x* in the range -π to π.
rand( )	Returns pseudo-random number *r*, where 0 <= *r* < 1.
srand(*x*)	Establishes new seed for **rand( )**. If no seed is specified, uses time of day. Returns the old seed.

Linux

Awk

Awk Function	Description
gsub(*r*,*s*,*t*)	Globally substitutes *s* for each match of the regular expression *r* in the string *t*. Returns the number of substitutions. If *t* is not supplied, defaults to **$0**.
index(*s*, *t*)	Returns position of substring *t* in string *s* or zero if not present.
length(*s*)	Returns length of string *s* or length of **$0** if no string is supplied.
match(*s*,*r*)	Returns either the position in *s* where the regular expression *r* begins, or 0 if no occurrences are found. Sets the values of **RSTART** and **RLENGTH**.
split(*s*, *a*, *sep*)	Parses string *s* into elements of array *a* using field separator *sep*; returns number of elements. If *sep* is not supplied, **FS** is used. Array splitting works the same way as field splitting.
sprintf("*fmt*", *expr*)	Uses **printf** format specification for **expr**.
sub(*r*,*s*,*t*)	Substitutes *s* for first match of the regular expression *r* in the string *t*. Returns 1 if successful; 0 otherwise. If *t* is not supplied, defaults to **$0**.
substr(*s*,*p*, *n*)	Returns substring of string *s* at beginning position *p* up to a maximum length of *n*. If *n* is not supplied, the rest of the string from *p* is used.
tolower(*s*)	Translates all uppercase characters in string *s* to lowercase and returns the new string.
toupper(*s*)	Translates all lowercase characters in string *s* to uppercase and returns the new string.